Computer Science Tripos

Part II Project Proposal Coversheet

Please fill in Part 1 of this form and attach it to the front of your Project Proposal.

Part 1

Name:	Kofi Wilkinson	CRSID:	ksw40
College:	Homerton	Project Checkers:(Initials)	A.B. S.K.
Title of Project:	Next Generation CHERI Tag Controller		
Date of submission:	16/10/2023	Will Human Participants be used?	No
Project Originator:	Dr Jonathan Woodruff		
Project Supervisor:	Dr Jonathan Woodruff		
Directors of Studies:	Dr John Fawcett		
Special Resource Sponsor:	Dr Jonathan Woodruff		
Special Resource Sponsor:			

Part 2

Project Checkers are to sign and comment in the students comments box on Moodle.

Part 3

For Teaching Admin use only

Date Received:		Admin Signature:	

Computer Science Tripos – Part II – Project Proposal

Next Generation CHERI Tag Controller

Kofi Wilkinson, Homerton College
Project Originator: Dr Jonathan Woodruff
16th October 2023

Project Supervisor: Dr Jonathan Woodruff

Director of Studies: Dr John Fawcett

Project Checkers: Prof. Alan Blackwell & Prof. Srinivasan Keshav

Introduction

Background

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing, they have been proven to provide system-wide security properties, such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5]. The paper continues to explain how single-bit tag (SBT) shadowspaces are sufficient to provide these properties, as one bit is enough to distinguish between data that has been manipulated by untrusted code, and data that retains the integrity of only being written by a program running through a trusted path of execution, and with this we can define as many new hardware-enforced types as we might need.
Section III of the paper describes four different approaches to implementing tagged memory. A direct quote of this list is as follows:

- *width* – Store tag bits in a wider physical memory, e.g., 65-bit words.
- *in-page* – Store tag bits in borrowed bytes of DRAM pages, slightly shrinking each DRAM page and skewing the DRAM address mapping.
- *dedicated* – Store tag bits in a dedicated memory.

- *table* – Store tag bits in a table in DRAM.

As the paper then explains, the *width* option is conceptually the simplest as its only unique requirement is to widen the system memory and buses. This approach has been taken before, e.g. by the Burroughs B6000 [6], however, power-of-two memory systems dominate today's industry, making this approach infeasible for an SBT architecture.
The *in-page* option leads to a fragmented tag shadowspace, limiting both tag cacheline size and spatial locality benefits. The *dedicated* option would require its own memory subsystem, complete with a dedicated interface and discrete chips, which brings too much design, area and power overhead for most applications.
This leaves the *table* option, and it is this that the rest of the paper concerns itself with optimising due to its practicality from having relatively little overhead. The overhead can be reduced by making the tag table more cacheable to decrease the frequency of suffering DRAM latency penalties. The tag table memory accesses and tag caching are all handled by a hardware tag controller, which determines the algorithm by which tags are read and written during execution.
Architectures that employ a tag table will statically allocate a region of DRAM at boot-time to act as the tag cache. This of course results in a static memory cost, which is typically acceptable for client applications, but unacceptable for server/cloud applications, according to feedback from industry. Different approaches of caching tags have been explored, which aim to reduce the DRAM tag table access frequency, but they do not address the constant DRAM space consumption overhead of the tag table itself. *Efficient Tagged Memory* details a hierarchical lookup scheme that compresses tags within the tag cache, thus storing more information about the tag table within the tag cache, further increasing tag cache hit rate, but it in fact has the effect of increasing the constant DRAM space consumption slightly due to the addition of a first-layer.

Project Focus

An idea that has not yet been explored comprehensively in literature is dynamic allocation of shadowspace for the tag table. Doing this would allow the following techniques to be used to reduce the tag table's DRAM space consumption:

- We could request only as much tag memory as is required for the current workload and free it back out to the rest of the system once it is no longer required.
- We could compress the tag table in DRAM, and then free the extra saved space.

One possibility is dynamic allocation of physical DRAM to different guest OSs running on one machine, managed by a hypervisor. How such a system might work is not the focus of my project, but mechanisms for dynamic allocation exist [7].
If a substantial amount of space can be saved by the two mentioned techniques, tag table implementations of memory, and subsequently tagged memory architectures, would take a significant step towards becoming viable for use in server/cloud applications, meaning that the aforementioned benefits they have provided to client applications can also be brought to these industries.

Starting Point

- I have read the paper *Efficient Tagged Memory*.
- I completed the Cambridge SystemVerilog Tutor [8] and performed the mandatory tasks 1 and 5 for the Part IB tick, providing me with some HDL experience with SystemVerilog. I also completed the optional task 2a, providing me with an introduction to HDL simulation with ModelSim.

- Over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm's Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model, and provided me with an understanding of how a CHERI tagged memory architecture might structure its pointers, and how tag shadowspace might be laid out and accessed in DRAM.

Substance and Structure

My project has several distinct but related objectives. The order in which I have listed the objectives is the order in which it makes sense to tackle them.

Throughout this project I will be focusing on SBT architectures due to their preference in industry and their simplicity. I will be focusing particularly on the CHERI architecture, as this is a speciality of the university and of my supervisor, and it is also the tagged memory architecture I am most familiar with, thanks to my aforementioned summer internship. Much of my work and findings however will be transferable to other tagged memory architectures.

Tag Controller Simulator and Algorithm Analysis (Core)

The **core** objective for my project is to develop a tag controller simulator that can be used to evaluate two existing algorithms for tag table storage. Given a workload, it should be able to determine the state of the tag cache and tag table at any point during program execution, the tag cache's hit ratio, the DRAM tag table access frequency and the overall execution stall time due to DRAM access latency.

I intend to develop the simulator in C++ for the following reasons:

- I am familiar with it from the IB Programming in C and C++ course and from contributing to the development of my IB group project in C++.
- It is a highly performant language which is important when parsing and simulating very large memory access traces.
- The traces are in a format which can be parsed very quickly and easily by existing C++ libraries.
- Lots of existing cache simulation tools are written in C++, and I will likely be integrating one of these into my tag controller simulator.

I will begin by programming a decoder that can parse the traces correctly and provide the simulator with the data it needs. Next, I will integrate in the cache simulator logic; such cache simulators do exist, and I will likely employ an existing one that has been verified to save unnecessary work, such as DnyamoRIO's drcachesim [9], allowing me to move forward to the interesting analysis stage which is the heart of the project. Once this is done and tested, I will begin working on analysis tools, which may include visualisations of the tag cache and/or tag table, or statistical properties of the memory accesses. After this, I will implement two existing baseline tag algorithms within the simulator. I intend to implement the hierarchical scheme detailed in *Efficient Tagged Memory*, and also Arm's Morello tag cache implementation. Evaluating these two is sensible due to my familiarity with both and the proven utility of both. These baselines may serve as a comparison point for a novel algorithm I develop later in the project.

Novel Tag Controller Algorithm (Extension 1)

As an **extension** objective, I aim to develop a novel algorithm for tag table storage. Hopefully the analysis tools will provide useful insight for me to use when developing the algorithm; I intend to use the findings from

the tools to inform the design, and also to investigate other possible techniques, such as the use of a tag cache prefetcher.

I will first need to devise a scheme for dynamically requesting and freeing tag shadowspace DRAM. Once this is done, I will explore how much memory can be saved by a basic algorithm that makes use of the dynamic allocation scheme. After this, I will develop a more advanced algorithm that uses the insights from the analysis tools and my investigation, and I will evaluate the performance of it against those of the two baseline algorithms. I will look at using existing prefetchers if I see them as useful rather than developing my own, as the aim of this extension is to devise a highly performant tag controller algorithm, and not to spend time developing my own prefetcher that likely would not compare against those that have already been optimised and verified.

Hardware Description Language Implementation (Extension 2)

As an **extension** objective, I aim to implement my algorithm in hardware using a hardware description language, check its correctness either by designing a suite of test cases or by formal verification, and evaluate its performance on different workloads.

I would use Bluespec SytemVerilog for this due to the language being designed for hardware implementations just like this and having a suite of tools, including simulators and FPGA synthesis tools, that I can leverage.

FPGA Synthesis (Extension 3)

As an **extension** objective, I aim to synthesise my implementation on an FPGA, check its correctness in a similar fashion, and evaluate its performance on different workloads.

Success Criteria

The core objective is to develop and check the correctness of the tag controller simulator, and to implement and evaluate the two aforementioned existing tag controller algorithms. The project will be successful if this objective is completed. To be specific, I will develop a program that, when given the parameters of the tag table and tag cache and a workload, meets the following criteria:

- it can simulate how the states of the tag table and tag cache change with each memory access of the workload, when employing two different existing algorithms for tag table storage
- for a simulation, it can determine the tag cache's hit ratio
- for a simulation, it can determine the DRAM tag table access frequency
- for a simulation, it can provide at least one other detail about the tag memory accesses that would be useful information to a professional analyst

Plan of Work

1) 16th Oct – 29th Oct

Work Tasks

- Obtain the traces and understand their format and how to parse them.
- Complete development of the trace decoder for the tag controller simulator.

Milestones

- Demonstrate a solid understanding of the format of the trace files by having a working trace decoder that is able to extract all relevant data from a given trace file.

2) 30th Oct – 12th Nov

Work Tasks

- Complete implementation of cache simulator logic.

Milestones

- Demonstrate familiarity with the chosen cache simulator by having it integrated into my tag controller simulator, and verified to be working correctly.

3) 13th Nov – 26th Nov

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

4) 27th Nov – 10th Dec

Work Tasks

- Complete the analysis/visualisation tools for the tag controller simulator.

Milestones

- Have checked with my supervisor that the tools would be of use to a professional by having received feedback on a formal document describing what the tools do. Have a finished suite of analysis/visualisation tools that provide useful information to the user.

5) 11th Dec – 24th Dec

Work Tasks

- Implement and evaluate the two existing algorithms using the simulator, and report all findings into a formal document.

Milestones

- Have emailed the document of my findings to my supervisor for review and iterated on any feedback received.

6) 25th Dec – 7th Jan

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II. I will also enjoy the Christmas holidays.

7) 8th Jan – 21st Jan

Work Tasks

- List all opportunities for improvement found, noting which ones I plan to implement in my algorithm and why. Also research techniques that might lead to an improved tag controller algorithm. Detail all my findings in a formal document.
- Make a final list of optimisations I aim to employ in my algorithm and develop a formal specification of my novel algorithm and get feedback from my supervisor.
- Implement the dynamic DRAM allocator into the tag controller simulator.

Milestones

- Have emailed my findings and the final list of optimisations to my supervisor for review, and have iterated on any feedback received.
- Have emailed the formal specification of the algorithm to my supervisor for review.
- Have confirmed with my supervisor that my implementation of the dynamic DRAM allocator does not make any false assumptions about real systems and is an accurate and useful model for the context in which my algorithm may run.

8) 22nd Jan – 4th Feb

Work Tasks

- Implement the algorithm in the tag controller simulator and evaluate its performance, recording all findings in a formal document.
- Write up my progress report, ask for a check from my supervisor if necessary, iterate on any feedback, and submit the final version.
- Prepare my progress report presentation.

Milestones

- Have implemented the algorithm, verified its correctness, evaluated its performance, and written up the formal document of my findings.
- Have completed and submitted my progress report.
- Have completed and rehearsed my progress report presentation to the point of being able to confidently deliver it in 5 minutes.

9) 5th Feb – 18th Feb

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II. I will also deliver my progress report presentation.

10) 19th Oct – 3rd Mar

Work Tasks

- Begin planning the HDL implementation of my algorithm.
- Pick a processor to adjoin the tag controller to, detailing the justification for my choice in a formal document.

- Begin programming the algorithm in my chosen HDL.

Milestones

- Have sent my plan to my supervisor for review and iterated on any feedback.

11) 4th Mar – 17th Mar

Work Tasks

- Finish the HDL implementation and verify that it correctly implements my tag controller algorithm, detailing the verification process in a formal document.
- Begin learning how to go about FPGA synthesis of the HDL implementation.
- Write the introduction and preparation sections of the write-up.

Milestones

- Have sent the formal document of verification details to my supervisor for review and confirmation, and iterated on any feedback.
- Have emailed the introduction and preparation sections to my supervisor and iterated on any feedback.

12) 18th Mar – 31st Mar

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

13) 1st Apr – 14th Apr

Work Tasks

- Learn all of the necessary tools/languages/frameworks required to synthesise the HDL implementation onto an FPGA.
- Begin attempting to synthesise the HDL implementation onto the FPGA and working through issues, contacting my supervisor if needed.
- Write the implementation section of the write-up.

Milestones

- Have had a discussion with my supervisor about how exactly to go about the FPGA synthesis.
- Have demonstrated my understanding and ability by synthesising an example netlist onto the FPGA I will use.
- Have emailed the implementation section to my supervisor and iterated on any feedback.

14) 15th Apr – 28th Apr

Work Tasks

- Successfully synthesise the HDL implementation onto an FPGA. Test the FPGA implementation and verify its correctness.

- Evaluate its performance and record all findings into a formal document. Compile this and all previous evaluation and conclusion documents and compose the evaluation and conclusion sections of the write-up.
- Compose the sections into one document that will be the final dissertation. Sent this to my director of studies for feedback, and also to my supervisor if significant changes to previously checked parts have been made.

Milestones

- Have a finalised report of the FPGA's performance when implementing my tag controller algorithm.
- Have emailed the evaluation and conclusion sections to my supervisor and iterated on any feedback.
- Have received feedback on my entire dissertation from my DoS and supervisor, and have iterated on that feedback. Have submitted the final version of my dissertation.

15) 29th Apr – 10th May

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

Resource Declaration

Extension 3 will require permission to use a suitable lab-owned FPGA.

Several memory access traces which I intend to use exist on the Computer Lab's internal network. To use them I will need access to the `/net/archive/export/cheri_traces` directory in which they reside.

My supervisor, Dr Jonathan Woodruff (jdw57), will be responsible for providing both of these resources.

For this project I will be using my laptop which has the following specifications:

- Intel i7 10th Generation Core
- 16 GB RAM
- Dual-boots Windows 10 and Ubuntu 22.04 LTS
- 1.5TB SSD storage

Should this device fail, I will be in need of a new laptop, so I will purchase an at-least equivalently capable machine immediately. I will be using a private GitHub remote repository to provide version control and backups of all my programs, files and documents.
I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.

References

[1] Alexandre Joannou et al. "Efficient tagged memory". In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.

[2] Brian E. Clark and Michael J. Corrigan. "Application System/400 performance characteristics". In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.

[3] Jedidiah R Crandall and Frederic T Chong. "Minos: Control data attack prevention orthogonal to memory model". In: *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE. 2004, pp. 221–232.

[4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.
[5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.
[6] Alastair JW Mayer. “The architecture of the Burroughs B5000: 20 years later and still ahead of the times?” In: *ACM SIGARCH Computer Architecture News* 10.4 (1982), pp. 3–10.
[7] *https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012 -r2-and-2012/hh831766(v=ws.11).*
[8] *http://www-ecad.cl.cam.ac.uk/.*
[9] *https://dynamorio.org/page_drcachesim.html.*